



Transações no MySQL

Transações

Concorrência e Transações no MySQL

O que é Concorrência no MySQL?

- **Concorrência** ocorre quando múltiplos processos ou transações acessam simultaneamente o banco de dados.
- O MySQL gerencia a concorrência para garantir a consistência dos dados, evitando conflitos e mantendo o desempenho.

- **Problemas típicos de concorrência:**

- **Leitura suja** (Dirty Read): uma transação lê dados de outra transação não confirmada.
- **Leitura não repetível** (Non-repeatable Read): uma transação lê dados que foram modificados por outra transação após a primeira leitura.
- **Phantom Read**: uma transação encontra um conjunto de dados diferente ao fazer a mesma consulta duas vezes.

Níveis de Isolamento de Transações no MySQL

O MySQL suporta quatro níveis de isolamento para gerenciar concorrência:

1. Read Uncommitted:

- Transações podem ler dados não confirmados.
- Possível ocorrência de leitura suja.

2. Read Committed:

- Apenas lê dados confirmados.
- Evita leitura suja, mas não evita leitura não repetível.

3. **Repeatable Read** (padrão do MySQL):

- As leituras feitas dentro de uma transação serão consistentes.
- Evita leitura suja e leitura não repetível.

4. **Serializable**:

- Transações são completamente isoladas.
- Mais seguro, mas reduz a concorrência, afetando o desempenho.

O que são Transações no MySQL?

- **Transação** é uma unidade de trabalho que garante consistência dos dados, mesmo em cenários de falha.
- Propriedades das transações (ACID):
 1. **Atomicidade**: todas as operações da transação devem ser realizadas com sucesso ou nada é feito.
 2. **Consistência**: uma transação leva o banco de dados de um estado consistente para outro.
 3. **Isolamento**: cada transação deve ser isolada das outras.
 4. **Durabilidade**: após a confirmação (commit), as alterações são permanentes, mesmo em caso de falha do sistema.

Comandos Básicos de Transações no MySQL

- Iniciar uma transação:

```
START TRANSACTION;
```

- **Confirmar (commit):** grava todas as alterações realizadas durante a transação.

```
COMMIT;
```

- **Reverter (rollback):** desfaz todas as operações realizadas durante a transação.

```
ROLLBACK;
```

- **Definir nível de isolamento:**

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Exemplo Prático: Transferência Bancária

Imagine que temos uma tabela de contas bancárias:

```
CREATE TABLE contas (  
  id INT PRIMARY KEY,  
  saldo DECIMAL(10,2)  
);
```

Transferir R\$100 da conta A para a conta B:

```
START TRANSACTION;
```

```
-- Debitar da conta A
```

```
UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
```

```
-- Creditar na conta B
```

```
UPDATE contas SET saldo = saldo + 100 WHERE id = 2;
```

```
COMMIT;
```

Se algo der errado (por exemplo, se a conta B não existir), podemos **reverter**:

```
ROLLBACK;
```

Concorrência com Transações

Imagine dois clientes tentando sacar dinheiro ao mesmo tempo da conta A:

1. Cliente 1: Verifica saldo e inicia saque.
2. Cliente 2: Verifica saldo e inicia saque antes de Cliente 1 confirmar a operação.

Solução com Transações e Nível de Isolamento:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
  
START TRANSACTION;  
  
-- Verificar saldo  
SELECT saldo FROM contas WHERE id = 1;  
  
-- Se saldo >= 100, realizar o saque  
UPDATE contas SET saldo = saldo - 100 WHERE id = 1;  
  
COMMIT;
```

O nível **Serializable** evita que Cliente 2 veja o saldo atualizado até que Cliente 1 confirme sua transação.

Exercícios Práticos

Exercício 1: Transferência com Concorrência

Crie duas conexões ao banco de dados MySQL e execute a seguinte sequência:

1. Na **Conexão 1**, inicie uma transação e debite o saldo de uma conta.
2. Na **Conexão 2**, antes de confirmar a transação da Conexão 1, tente debitar o saldo da mesma conta.
3. Teste os resultados com diferentes níveis de isolamento.

Exercício 2: Simulando Leitura Não Repetível

1. Inicie uma transação e faça uma leitura de saldo de uma conta.
2. Em outra transação, atualize o saldo dessa conta e confirme.
3. Na primeira transação, tente ler novamente o saldo e veja o comportamento.

Opções de Níveis de Isolamento no MySQL:

Para alterar o nível de isolamento no MySQL, você pode usar o comando `SET TRANSACTION ISOLATION LEVEL`. Esse comando pode ser usado de duas maneiras, dependendo de como você deseja definir o nível de isolamento: globalmente para todo o servidor ou para uma transação específica.

1. **READ UNCOMMITTED**
2. **READ COMMITTED**
3. **REPEATABLE READ** (padrão do MySQL)
4. **SERIALIZABLE**

1. Alterar o nível de isolamento para uma transação específica

Você pode definir o nível de isolamento **apenas para a transação atual**. Isso permite que você controle o comportamento de concorrência transação por transação.

Exemplo:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Em seguida, você inicia a transação:

```
START TRANSACTION;
```

```
-- Sua lógica de transação
```

```
COMMIT;
```

2. Alterar o nível de isolamento globalmente (para todas as transações)

Se você quiser alterar o nível de isolamento para todas as novas transações (por exemplo, após um reinício do servidor), use o comando abaixo para defini-lo globalmente:

```
SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Isso afeta todas as novas conexões e transações. As transações que já estavam em andamento antes da mudança continuam com o nível de isolamento anterior.

3. Verificar o nível de isolamento atual

Você pode verificar o nível de isolamento atual de uma sessão ou de todo o servidor:

- Para verificar o nível de isolamento da sessão atual:

```
SELECT @@SESSION.tx_isolation;
```

- Para verificar o nível de isolamento global:

```
SELECT @@GLOBAL.tx_isolation;
```

4. Exemplo completo de mudança de isolamento

1. Mudar o nível de isolamento para **READ COMMITTED** em uma transação específica:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
START TRANSACTION;  
  
-- Exemplo de consulta ou atualização  
SELECT * FROM contas WHERE id = 1;  
  
COMMIT;
```

2. Mudar o nível de isolamento globalmente para **SERIALIZABLE**:

```
SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Observações:

- Alterar o nível de isolamento global afeta todas as novas transações, mas não as transações em andamento.
- Você pode alterar o nível de isolamento em uma transação específica para atender às necessidades de isolamento e concorrência sem afetar o comportamento global do servidor. Essas configurações ajudam a equilibrar **performance** e **consistência de dados** conforme a necessidade da aplicação.

Vantagens e Desvantagens dos Níveis de Isolamento

Read Uncommitted:

- **Vantagens:** alta performance.
- **Desvantagens:** maior risco de inconsistência.

Serializable:

- **Vantagens:** máxima segurança e consistência.
- **Desvantagens:** redução significativa de concorrência e desempenho.

Repeatable Read (padrão):

- **Vantagens:** equilíbrio entre segurança e performance.
- **Desvantagens:** pode causar lock de registros por longos períodos.

Conclusão

- Transações são fundamentais para garantir a integridade dos dados em operações complexas.
- A escolha do **nível de isolamento** impacta diretamente o desempenho e a consistência dos dados.
- Compreender e aplicar corretamente os níveis de concorrência e transações no MySQL é crucial para evitar problemas como **leitura suja** e **leitura não repetível**.

Exemplo de Exercícios Práticos:

Exercício 1: Transação de Transferência entre Contas

- Crie uma Stored Procedure que execute uma transferência entre duas contas. A transação deve ser atômica: se houver erro (por exemplo, saldo insuficiente), a operação completa deve ser revertida.

Exemplo:

```
DELIMITER //
CREATE PROCEDURE TransferirValores(
    IN origem INT,
    IN destino INT,
    IN valor DECIMAL(10,2)
)
BEGIN
    DECLARE saldo_origem DECIMAL(10,2);

    -- Iniciar a transação
    START TRANSACTION;

    -- Verificar saldo da conta de origem
    SELECT saldo INTO saldo_origem FROM contas WHERE id = origem;

    IF saldo_origem < valor THEN
        -- Se saldo insuficiente, reverter a transação
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Saldo insuficiente';
    ELSE
        -- Debitar da conta de origem
        UPDATE contas SET saldo = saldo - valor WHERE id = origem;

        -- Creditar na conta de destino
        UPDATE contas SET saldo = saldo + valor WHERE id = destino;

        -- Confirmar a transação
        COMMIT;
    END IF;
END //
DELIMITER ;
```

Exercício 2: Trigger para Impedir Saques Negativos

- Crie um Trigger que impede saques se o saldo for insuficiente.

Exemplo:

```
CREATE TRIGGER verificar_saldo_antes_saque
BEFORE UPDATE ON contas
FOR EACH ROW
BEGIN
    IF NEW.saldo < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Saque não permitido. Saldo insuficiente';
    END IF;
END;
```

Exercício 3: Concorrência com Nível de Isolamento

- Crie duas transações concorrentes que tentam sacar o mesmo valor de uma conta e observe o comportamento com diferentes níveis de isolamento (ex: **Read Committed** vs **Serializable**).